Gloss to Text Module

Gloss to Text Module

Where this fits in the project

The full system has three parts: the camera captures hand movements, the recognition model (MLP for static letters, LSTM for words) turns that into a list of signed words, and this module takes that list and turns it into a real sentence before it's shown to the user. In the architecture diagram, this is the "NLP Mapping" step, right before the response gets sent back to the app.

What this does

Our sign recognition model outputs raw words in ASL order, without grammar. For example, if someone signs "I want to eat an apple", the model gives us:

```
["I", "want", "eat", "apple"]
```

This is correct ASL grammar, but it reads strangely for anyone who doesn't know ASL. This module takes that word list and turns it into a normal English sentence:

```
"I want to eat an apple."
```

It does this by sending the word list to a language model with strict instructions: only add small connector words (a, the, to, is…), never add new meaning, and never drop or change any of the original words. If the model output doesn't match those rules, or if the model isn't reachable at all, the code falls back to a simpler rule-based version so the sentence still comes out instead of crashing.

How it works

1. **Check the words** – Every word coming in is checked against our known vocabulary list (the same words our recognition model was trained on). If a word isn't recognized, it gets flagged as a warning instead of silently ignored. This matters because if the recognition model misfires and sends a word we never trained on, we want to know about it rather than have it quietly show up in a sentence.

2. **Ask the model** – The word list is sent to a local model running through Ollama, along with a fixed system prompt. That prompt is the actual core of the module. It tells the model:

   - Output one sentence only, nothing else
   - It can only add small connector words (a, the, is, to, will…)
   - It cannot add any new noun, verb, or fact that wasn't in the original list
   - It cannot drop any word from the original list
   - Emotion words (sad, happy, angry, love, sick, tired…) must stay exactly as signed, not softened or exaggerated
   - If the gloss looks like a question (has words like where, who, how), answer with a question

   This is what stops the model from "getting creative" and adding things nobody signed.

3. **Check the answer** – Before trusting the model's sentence, the code checks that every original word is still represented in it (using basic stemming, so "eat" also matches "eating" or "eaten"). If a word got dropped, or the model added extra unrelated content, or it returned more than one sentence, the answer is rejected.

4. **Fallback if needed** – If the model isn't running, or its answer didn't pass the check, a basic rule-based sentence builder kicks in instead. It capitalizes "I", adds "to" after "want" when followed by a verb, and adds a question mark if a question word is present. It's not as smooth as the LLM output, but it always produces something usable, so the demo never breaks.

Example runs

Here's what the module produces for a few different gloss sequences:

Gloss input	Local LLM output	Rule-based fallback
`["I", "want", "eat", "apple"]`	"I want to eat an apple."	"I want to eat apple."
`["I", "happy", "meet", "friend"]`	"I am happy to meet my friend."	"I happy meet friend."
`["where", "brother", "go"]`	"Where did my brother go?"	"Where brother go?"
`["I", "sad", "sick", "headache"]`	"I am sad because I am sick with a headache."	"I sad sick headache."
`["thank", "you", "help", "me"]`	"Thank you for helping me."	"Thank you help me."

The LLM output is noticeably more natural. The fallback is rougher but keeps every original word and never crashes.

Why Ollama instead of a paid API

We're using Ollama so the model runs on our own machine instead of calling a paid service. No API key, no cost, no internet needed once the model is downloaded. The tradeoff is that local models are smaller and occasionally less accurate, which is exactly why the fallback and the answer-checking step exist.

Setup

1. Install Ollama: https://ollama.com/download
2. Pull a model:

   ```
   ollama pull phi3:mini
   ```

   (or `llama3.1:8b` for better quality if your machine can handle it)
3. Install the Python package:

   ```
   pip install ollama
   ```
4. Run the file:

   ```
   python3 gloss_to_text_ollama.py
   ```

What's in the file

- `TARGET_FOLDERS` / `VOCAB_SET` – the list of words our model was trained on, used for validation.
- `GlossToTextResult` – a small object that holds the result: the original gloss, the final sentence, which method produced it, a confidence score, and any warnings.
- `validate_gloss()` – checks incoming words against the vocabulary.
- `SYSTEM_PROMPT` / `build_user_prompt()` – the instructions and message format sent to the model.
- `_call_ollama()` – sends the request to the local model and returns its answer, or `None` if anything goes wrong.
- `sanity_check()` – makes sure the model's answer didn't drop or add content.
- `rule_based_fallback()` – the simple backup sentence builder.
- `gloss_to_text()` – the main function that ties all of the above together. This is the only function the rest of the app needs to call.

Using it in code

```python
from gloss_to_text_ollama import gloss_to_text

result = gloss_to_text(["I", "want", "drink", "coffee"])

print(result.sentence)   # the final sentence
print(result.method)     # "llm_local" or "rule_based_fallback"
print(result.warnings)   # any issues found along the way
```

This is the piece that plugs into the rest of the pipeline: the recognition model produces the word list, this module turns it into a sentence, and that sentence gets sent back to the app to display.

Known limitations

- Smaller models sometimes break the rules (adding words, making two sentences), which is why the fallback exists. The smaller the model, the more often this happens.
- The vocabulary check only flags unknown words, it doesn't stop the pipeline. Handling that is a decision the app layer should make.
- The rule-based fallback is intentionally simple. It's a safety net, not meant to be the main output.
- Local models are slower than a cloud API, especially on a CPU-only machine. On weaker hardware, expect a few seconds per sentence.
- The word-matching in the sanity check is based on simple stemming, not real grammar understanding, so it can occasionally flag a correct sentence as wrong (a false rejection), which just means the fallback gets used more often than it strictly needs to.

Possible improvements later

- Try a mid-sized model like `qwen2.5:7b` for a better balance of speed and accuracy if the team's machines can handle it.
- Log every fallback case during testing to see which gloss patterns the small model struggles with most, and adjust the prompt accordingly.
- Cache repeated gloss sequences so common phrases don't need to hit the model every time.